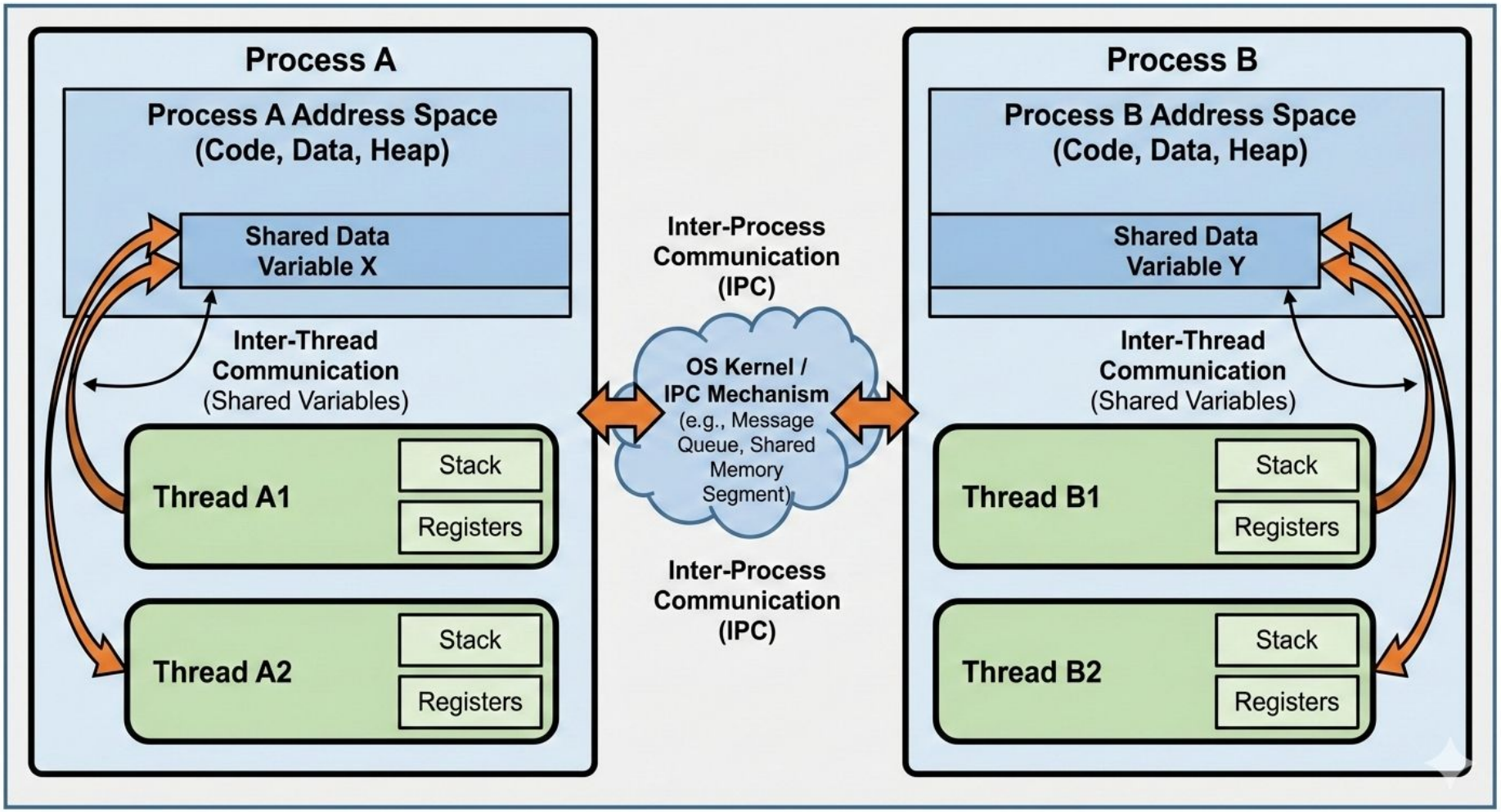
Chapter 5

Multithreading and Parallel Programming and Concurrency

Course repository:
https://github.com/skanderturki/se411

Thread Vs Process



Thread Vs Process

Feature	Process	Thread
Definition	An executing instance of a program. It's a self-contained execution environment.	The smallest unit of execution within a process. A process can contain multiple threads.
Memory	Has its own, isolated address space (code, static data, dynamic heap).	Shares the address space of its parent process with other threads.
Resources	Owns resources like file handles, sockets, and device contexts.	Shares the process's resources.
Overhead	"Heavyweight." Creating and switching between processes is slower and consumes more resources.	"Lightweight." Creating and switching between threads is faster and more efficient.
Isolation	High isolation. A crash in one process typically doesn't affect others.	Low isolation. A bug or crash in one thread can crash the entire process.
Communication	Requires Inter-Process Communication (IPC) mechanisms managed by the OS (e.g., pipes, sockets, message queues).	Communicates directly by reading and writing shared variables in the process's memory.

The **heap** is a region of a process's memory used for **dynamic memory allocation**. This is where your objects, arrays are stored.

The **static data** (in Java called metaspace) region is where class and method names and static and global variables are stored.

Both are shared between threads of the process.

Threads Concept

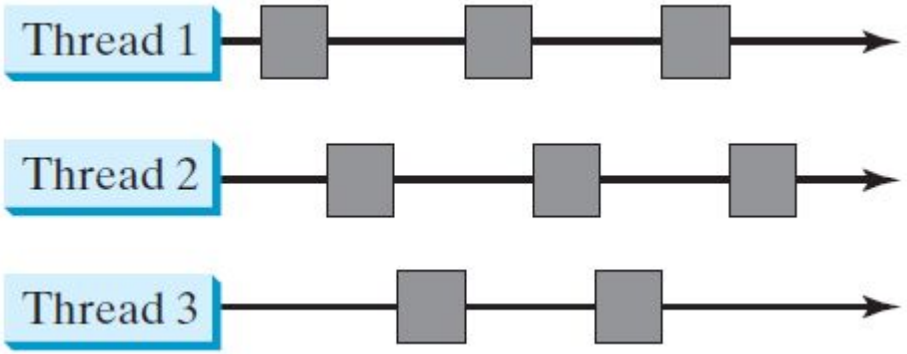
Multiple threads on multiple CPUs

Multi-processing (parallelism)

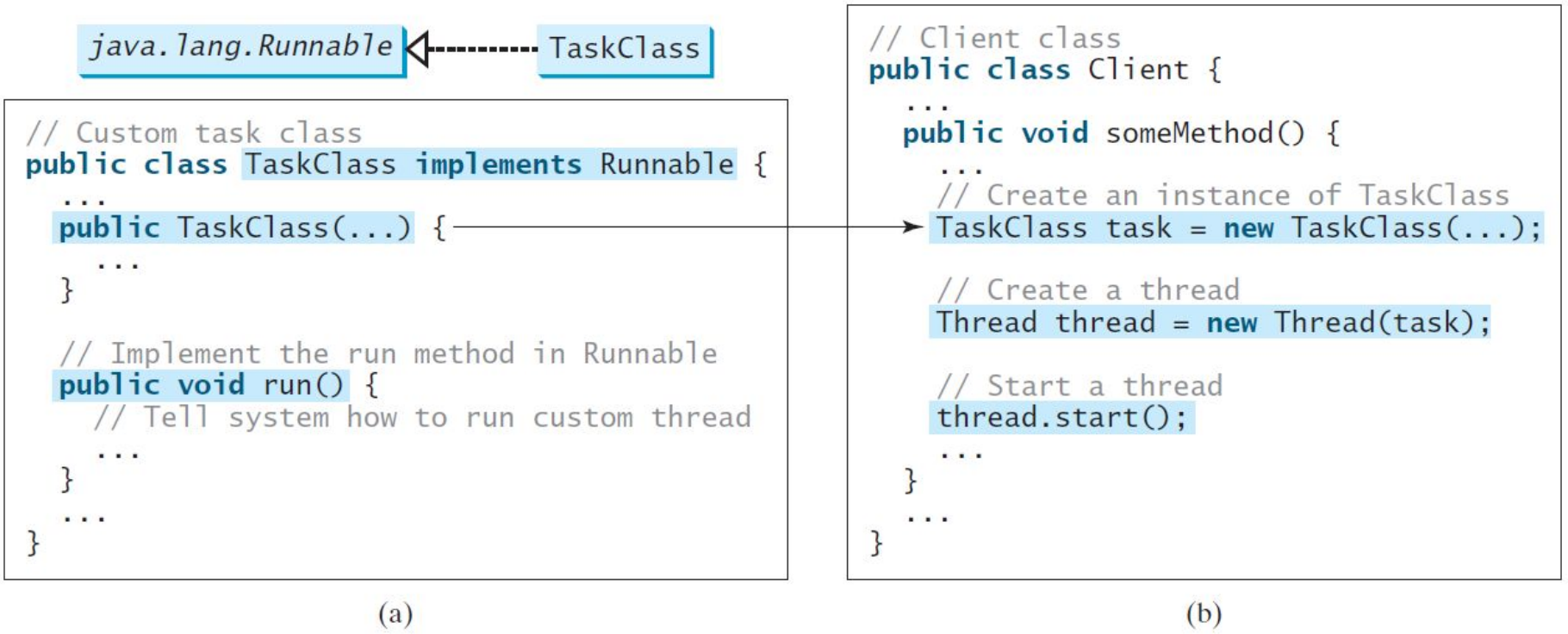


Multiple threads sharing a single CPU

Multi-threading (concurrency / time slicing)



Creating Tasks and Threads



Example: Using the `Runnable` Interface to Create and Launch Threads

- Objective: Create and run two threads:
  - The first thread counts with a frequency of 20Hz.
  - The second thread counts with a frequency of 10Hz.

Example 01 (1)

Course repository: ./ch_05/01_intro

```
package edu.psu.se411.ch05.threads;

public class AdditionTask implements Runnable {
     private int counter;
     private String taskId;
     private int frequency;     // times per second

     public AdditionTask(String id_p, int frequency_p) {
          this.frequency = frequency_p;
          this.taskId = id_p;
     }

     @Override
     public void run() {
        while(true) {
          Long period = 1000 / frequency;
          counter++;
          System.out.printf("Thread %s Added a and now count is %d%n",
                getTaskId(), getCounter());
          try {
             Thread.sleep(period); // frequency in milliseconds
          } catch (InterruptedException e) {
             Thread.currentThread().interrupt();
             break;
          }

        }
     }
}
```

Example 01 (2)

```
package edu.psu.se411.ch05;

import edu.psu.se411.ch05.threads.AdditionTask;

public class App {
     static Logger logger = LoggerFactory.getLogger(App.class);


     public static void main(String[] args) {
          logger.info("Application is starting...");


          AdditionTask addTaskA = new AdditionTask ("Task_A", 4);
          AdditionTask addTaskB = new AdditionTask ("Task_B", 10);

          Thread thread_A = new Thread(addTaskA);
          Thread thread_B = new Thread(addTaskB);

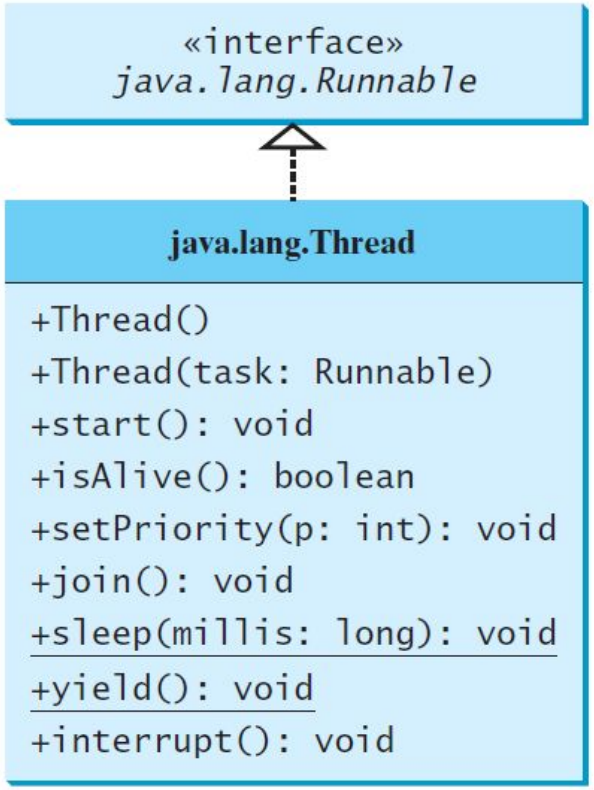
          thread_A.start();
          thread_B.start();

          logger.info("Application is closing...");
     }


}
```

Note: Your two AdditionTask objects (addTaskA and addTaskB) live in the **process heap**, not inside the threads.

The Thread Class



Creates an empty thread.	
Creates a thread for a specified task.	
Starts the thread that causes the `run()` method to be invoked by the JVM.	
Tests whether the thread is currently running.	
Sets priority `p` (ranging from 1 to 10) for this thread.	
Waits for this thread to finish.	
Puts a thread to sleep for a specified time in milliseconds.	
Causes a thread to pause temporarily and allow other threads to execute.	
Interrupts this thread.	

The Static yield() Method

You can use the yield() method to temporarily release time for other threads. For example:

```
public void run() {
  for (int i = 1; i <= lastNum; i++) {
    System.out.print(" " + i);
    Thread.yield();
  }
}
```

Every time a number is printed, the thread is yielded. So, other threads can continue executing.

The Static sleep(milliseconds) Method

The sleep(long mills) method puts the thread to sleep for the specified time in milliseconds. For example, suppose you modify the previous example as follows:

```
public void run() {
    while(true) {
        counter++;
        System.out.printf("Task %s added 1, now count is %d%n", getTaskId(),
        getCounter());
        try {
          Thread.sleep(1000 / frequency);
        } catch (InterruptedException e) {
          System.out.printf("Task %s was interrupted at count %d%n“
             , getTaskId(), getCounter());
        }
    }
}
```

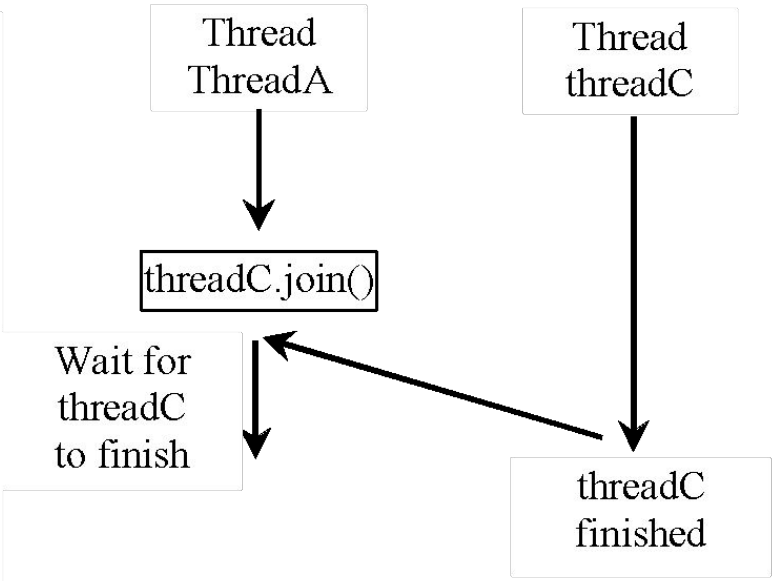
Every time a number is printed, the thread is put to sleep for 1/frequency seconds.

The join() Method

Course repository: ./ch_05/02_join

You can use the join() method to force one thread to wait for another thread to finish. For example:

```
public void run() {
  Thread threadC = new Thread(
    new AdditionTask(40));
  threadC.start();
  try {
    for (int i = 1; i <= lastNum; i++) {
      System.out.print(" " + i);
      if (i == 50) threadC.join();
    }
  }
  catch (InterruptedException ex) {
  }
```



The numbers after 50 are printed after thread threadC is finished.

Implement Runnable Vs Extend Thread

```
public class SomeOpRunnable implements Runnable {

    private Double x;
    private String opName;

    public SomeOpRunnable(String opName, Double x) {
        super();
        this.x = x;
        this.opName = opName;
    }

    public void count() {
        for(int i = 0; i < 1000000000; i++) {
            x ++;
            if(x % 10000000 == 0)
                System.out.printf("%s counting at: %f%n", opName, x);
        }
    }

    @Override
    public void run() {
        count();
    }
```

Then, in the client code:



```
SomeOpRunnable opA = new SomeOpRunnable("OpA", 405.0);
SomeOpRunnable opB = new SomeOpRunnable("OpB", 80.0);

Thread opAThread = new Thread(opA);
Thread opBThread = new Thread(opB);

opAThread.start();
opBThread.start();
```

Implement Runnable Vs Extend Thread (2)

```
public class SomeOpThread extends Thread {

    private Double x;
    private String opName;

    public SomeOpThread(String opName, Double x) {
        super();
        this.x = x;
        this.opName = opName;
    }

    @Override
    public void run() {
        count();
    }

    public void count() {
        for(int i = 0; i < 1000000000; i++) {
            x ++;
            if(x % 10000000 == 0)
                System.out.printf("%s counting at: %f%n", opName, x);
        }
    }
```

Then, in the client code:



```
SomeOpThread opA = new SomeOpThread("OpA", 405.0);
SomeOpThread opB = new SomeOpThread("OpB", 80.0);

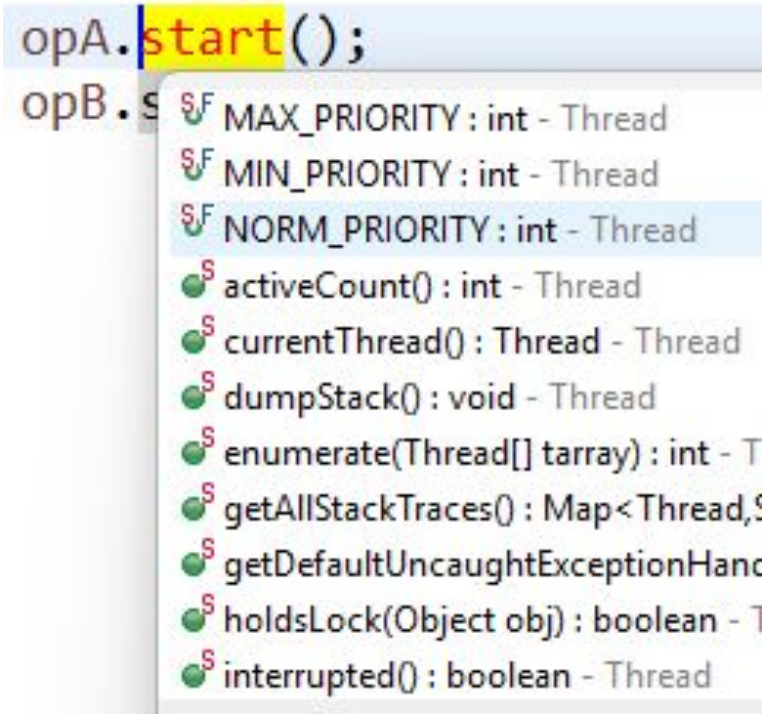
opA.start();
opB.start();
```

Implement Runnable Vs Extend Thread (3)

With the Thread extension approach, you can have:

+ cleaner client code

- but you pollute the class with all the Thread methods:



Thread Priority

- Each thread is assigned a default priority of `Thread.NORM_PRIORITY`. You can reset the priority using `setPriority(int priority)`.

- Some constants for priorities include
  `Thread.MIN_PRIORITY`
  `Thread.MAX_PRIORITY`
  `Thread.NORM_PRIORITY`

```
public class Thread implements Runnable {

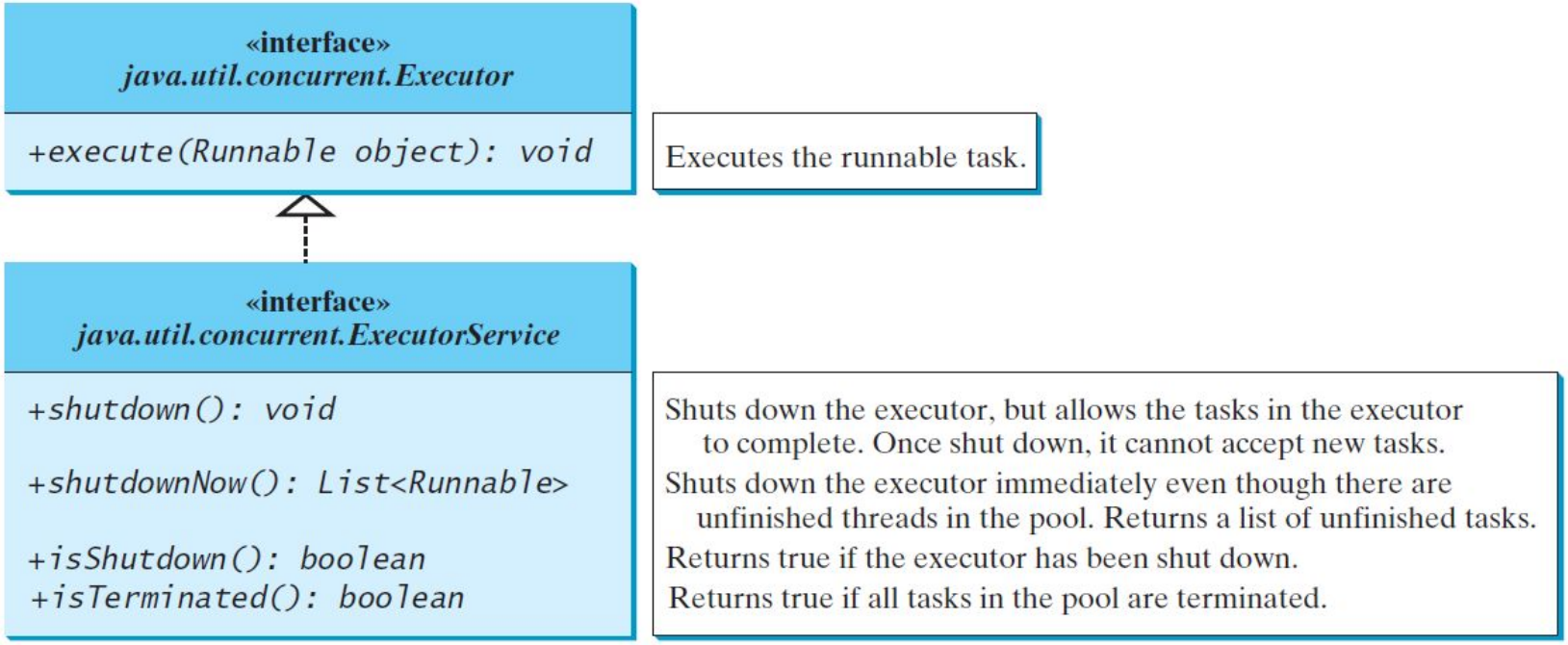
  /* Some private helper methods */
  private native void setPriority0(int newPriority);
```



Implemented at the level of the JVM (its implementation depends on the operating system), for example: if the OS does not support thread priorities, the implementation would be empty.

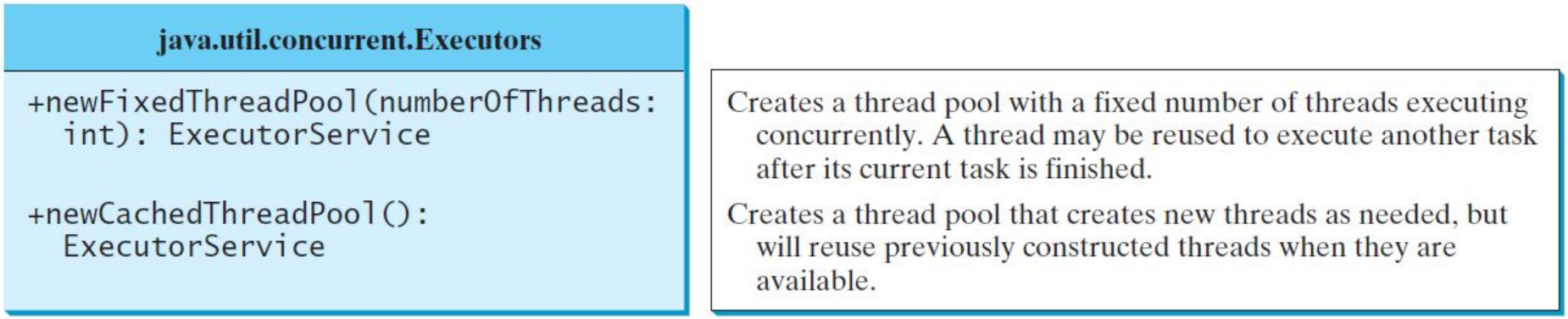
Thread Pools

Starting a new thread for each task could limit throughput and cause poor performance. A thread pool is ideal to manage a number of tasks executing concurrently. JDK 1.5 uses the Executor interface for executing tasks in a thread pool and the ExecutorService interface for managing and controlling tasks. ExecutorService is a subinterface of Executor.



Creating Executors

To create an Executor service object, use the **static** methods in the Executors class.



Example 04: Executor

Course repository: ./ch_05/04_executor

```
public static void main(String[] args) {

  ExecutorService executor = Executors.newFixedThreadPool(3);

  for(int i = 0; i < 20; i++) {
    CountingTask countingTask = new CountingTask("Task_" + i, 20, 20);

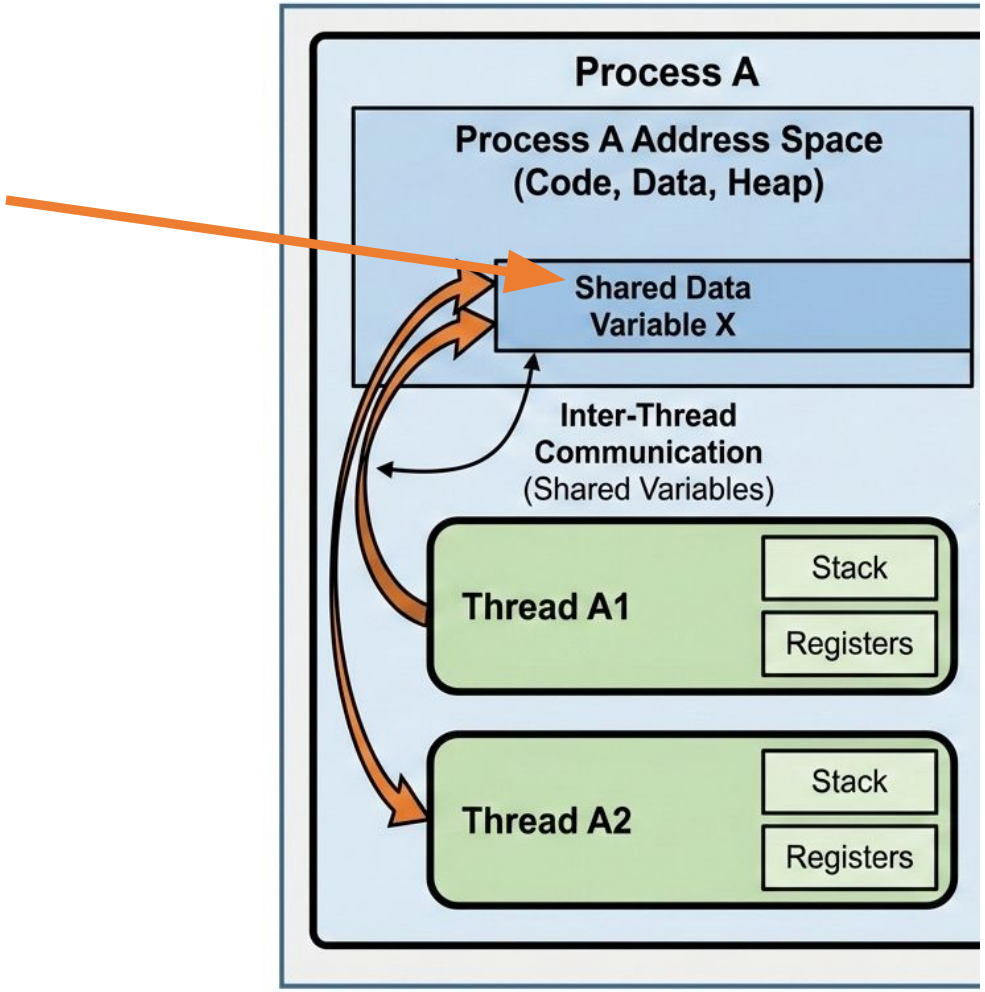
    // Submit runnable tasks to the executor
    executor.execute(countingTask);
  }

  // Shut down the executor
  executor.shutdown();
}
```

Note: 20 threads will be executed but only three at a time.

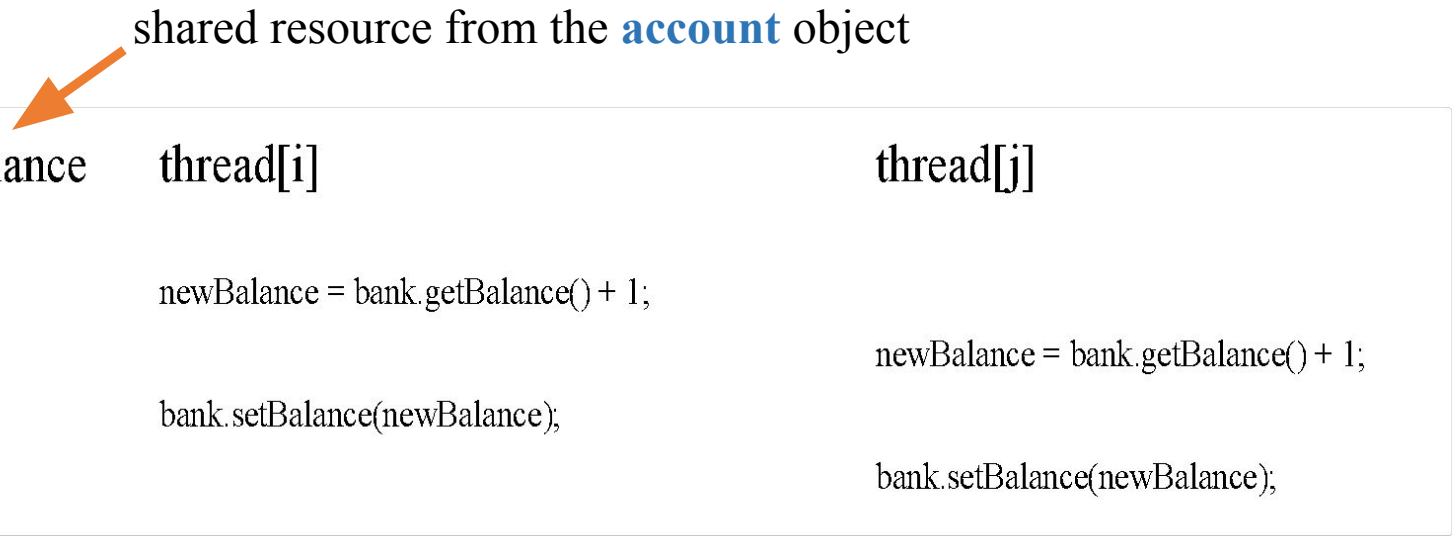
Threads and concurrency

Threads compete to get read and/or write access to process variables



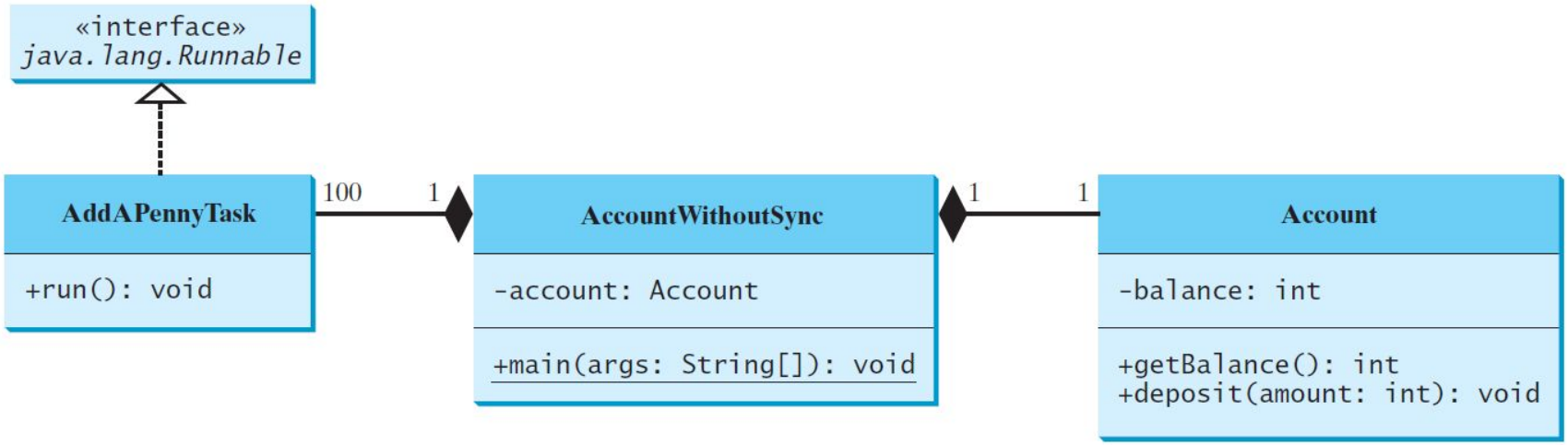
Thread Synchronization

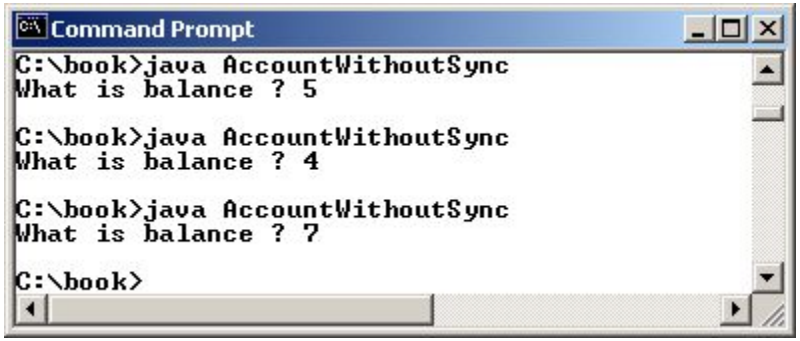
Problem: A shared resource may be corrupted if it is accessed simultaneously by multiple threads. For example, two unsynchronized threads accessing the same bank account may cause conflict.



Example: Showing Resource Conflict

Objective: Write a program that demonstrates the problem of resource conflict. Suppose that you create and launch one hundred threads, each of which adds a penny to an account. Assume that the account is initially empty.





Example: Resource conflict

Course repository: ./ch_05/05_bank_account

```
 private static Account account;

 public static void main(String[] args) {
   ExecutorService executor = Executors.newCachedThreadPool();

   // Create and launch 100 threads
   for (int i = 0; i < 100; i++) {
     executor.execute(new AddAPennyTask());
   }

   executor.shutdown();

   // Wait until all tasks are finished
   while (!executor.isTerminated()) {
   }

   System.out.println("What is balance? " + account.getBalance());
 }


// A Runnable (or task) for adding a penny to the account
private class AddAPennyTask implements Runnable {
  public void run() {
    account.deposit(1);
  }
}
```

Example: Resource conflict (2)

Course repository: ./ch_05/05_bank_account

```
// A class for account
private class Account {
  private int balance = 0;


  public int getBalance() {
    return balance;
  }


  public void deposit(int amount) {
    int newBalance = balance + amount;

    // This delay is deliberately added to magnify the
    // data-corruption problem and make it easy to see.
    try {
      Thread.sleep(1);
    }
    catch (InterruptedException ex) {
    }

    balance = newBalance;
  }
}
```

Note: in this example, "balance" is the **critical resource** that is read and modified by different threads. But between the **reading** and the **writing** many other threads have also read the "old" value.

Race Condition

What, then, caused the error in the example? Here is a possible scenario:

Step	balance	Task 1	Task 2
1	0	newBalance = balance + 1;	
2	0		newBalance = balance + 1;
3	1	balance = newBalance;	
4	1		balance = newBalance;

The effect of this scenario is that Task 1 did nothing, because in Step 4 Task 2 overrides Task 1's outcome. Obviously, the problem is that Task 1 and Task 2 are accessing a common resource in a way that causes conflict. This is a common problem known as a ***race condition*** (حالة تنافس) in multithreaded programs.

A class is said to be ***thread-safe*** if an object of the class does not cause a race condition in the presence of multiple threads. As demonstrated in the preceding example, the Account class is not thread-safe.

The `synchronized` keyword

To avoid **race conditions**, more than one thread must be prevented from simultaneously entering certain parts of the program, known as **critical regions**.

The **critical region** in the previous example is the entire **deposit** method. There are several ways to correct the problem, one approach is to make Account thread-safe by adding the synchronized keyword in the deposit method.

By adding the **synchronized** keyword to the method, you protect it from concurrent access (only one thread can execute the method at a time), as follows:

```
public synchronized void deposit(double amount) { …. }
```

Synchronizing Instance Methods and Static Methods

A **synchronized method** acquires a **lock** before it executes.

In the case of an **instance method**, the lock is on the **object** for which the method was invoked.

In the case of a **static method**, the lock is on the class.

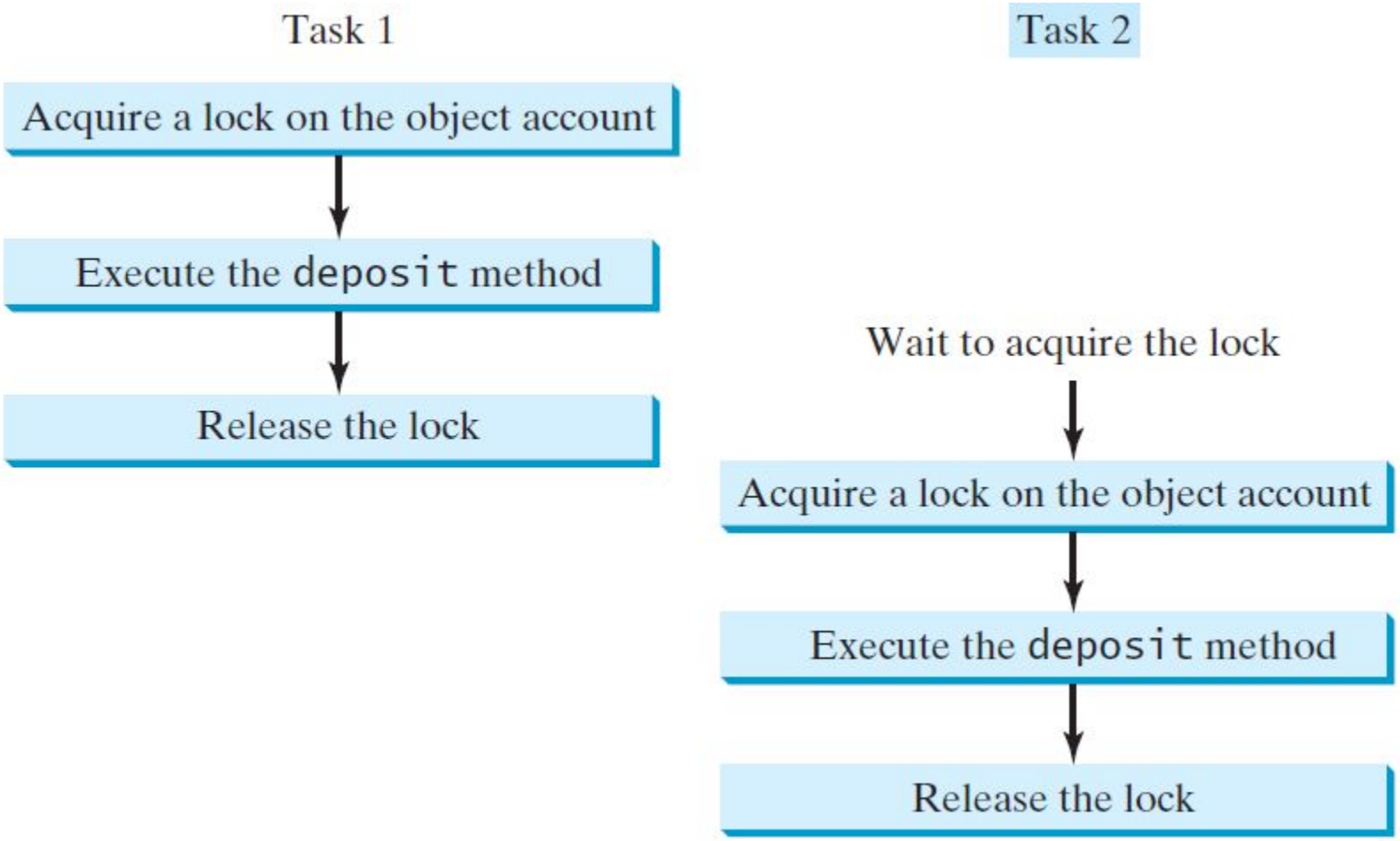
If one thread invokes a synchronized instance method (respectively, static method) on an object, the lock of that object (respectively, class) is acquired first, then the method is executed, and finally the lock is released. Another thread invoking the same method of that object (respectively, class) is blocked until the lock is released.

Synchronizing Instance Methods and Static Methods

With the deposit method synchronized, the preceding scenario cannot happen. If Task 2 tries to execute the method, and Task 1 is already in the method, Task 2 is blocked until Task 1 exits the method.

Step	Balance	Task 1	Task 2
1	0	`newBalance = balance + 1;`	
2	0		`newBalance = balance + 1;`
3	1	`balance = newBalance;`	
4	1		`balance = newBalance;`

Synchronizing Tasks




Thread
Thread
Critical Resource
Lock

Synchronizing Statements

Invoking a synchronized instance method of an object acquires a lock on the object, and invoking a synchronized static method of a class acquires a lock on the class.

A **synchronized statement** can be used to acquire a **lock on an object** and execute a block of code. This block is referred to as a *synchronized block*. The general form of a synchronized statement is as follows:

```
synchronized (expr) {
  statements;
}
```

The expression expr must evaluate to an object reference. If the object is already locked by another thread, the thread is blocked until the lock is released. When a lock is obtained on the object, the statements in the synchronized block are executed, and then the lock is released.

Synchronizing Statements vs. Methods

Any synchronized instance method can be converted into a synchronized statement. Suppose that the following is a synchronized instance method:

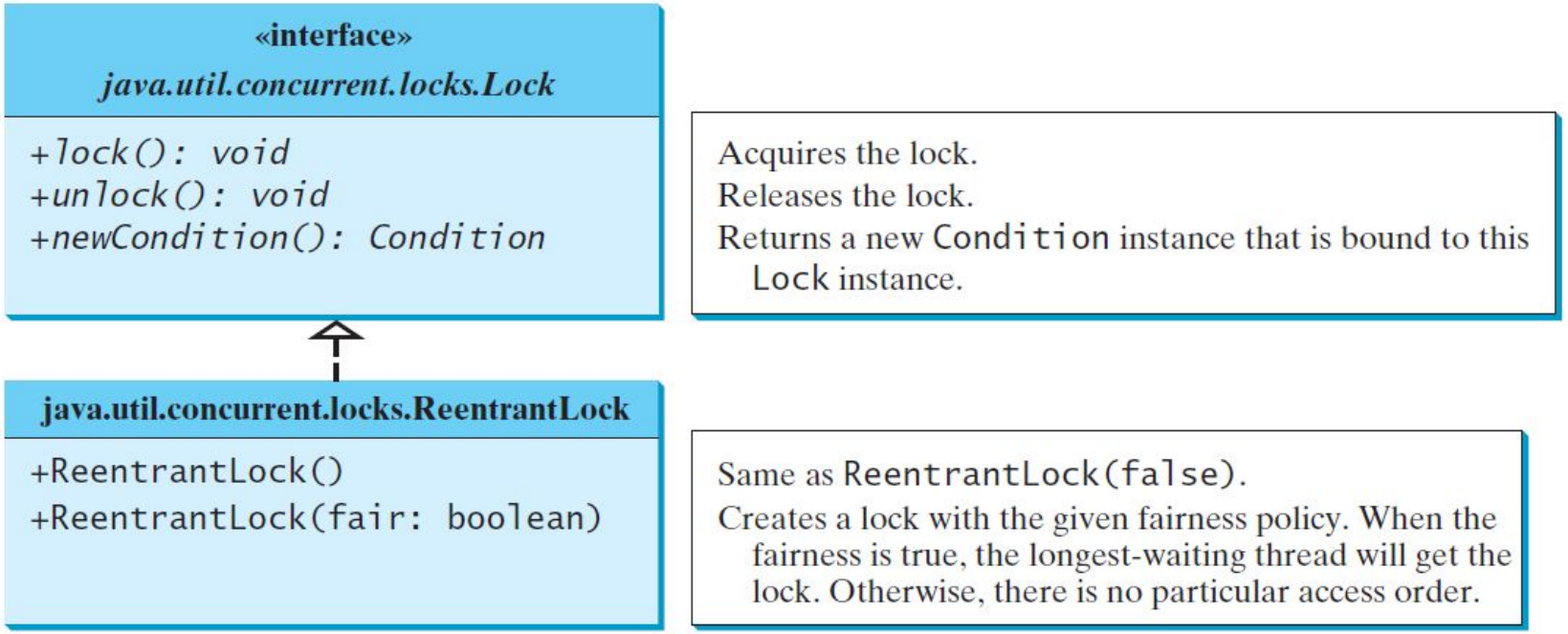
```
public synchronized void xMethod() {
  // method body
}
```

This method is equivalent to

```
public void xMethod() {
  synchronized (this) {
    // method body
  }
}
```

Synchronization Using Locks

A synchronized instance method implicitly acquires a lock on the instance before it executes the method.
JDK 1.5 (or 5, since 2004) enables you to use locks explicitly. These locking features are flexible and give you more control for coordinating threads. A lock is an instance of the Lock interface, which declares the methods for acquiring and releasing locks.



Fairness Policy

ReentrantLock is a concrete implementation of Lock for creating mutual exclusive locks. You can create a lock with the specified fairness policy. If fairness policy is set to true, the longest-waiting thread will obtain the lock first.

If fairness policy is set to false, the lock is granted to a waiting thread without any access order. Programs using fair locks accessed by many threads may have poor overall performance than those using the default setting, but have smaller variances in times to obtain locks and guarantee lack of **starvation** (a thread waiting to acquire a lock but never gets it).

Example: Using Locks

This example revises the bank account example to synchronize the account modification using explicit locks.

```
private static Account account = new Account();

public static void main(String[] args) {
  ExecutorService executor = Executors.newCachedThreadPool();

  // Create and launch 100 threads
  for (int i = 0; i < 100; i++) {
    executor.execute(new AddAPennyTask());
  }
  executor.shutdown();

  // Wait until all tasks are finished
  while (!executor.isTerminated()) {}

  System.out.println("What is balance ? " + account.getBalance());
}

// A Runnable for adding a penny to the account
public class AddAPennyTask implements Runnable {
  public void run() {
    account.deposit(1);
  }
}
```

Example: Using Locks (2)

```
// A class for account
public class Account {
  private Lock lock = new ReentrantLock(); // Create a lock
  private int balance = 0;

  public Double getBalance() {
    lock.lock();
    try {   return balance;  } finally {
       lock.unlock();
    }
  }
  public void deposit(int amount) {
    lock.lock(); // Acquire the lock

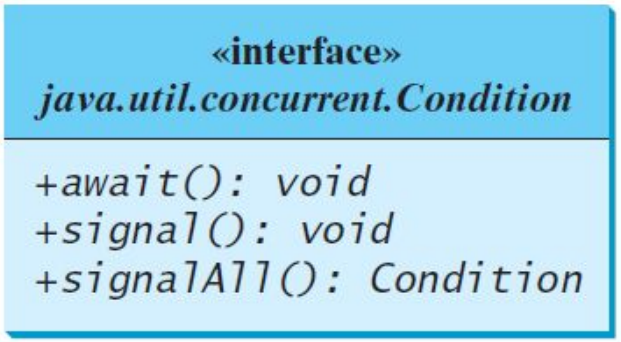
    try {
      int newBalance = balance + amount;

      // This delay is deliberately added to magnify the
      // data-corruption problem and make it easy to see.
      Thread.sleep(5);

      balance = newBalance;
    } catch (InterruptedException ex) {
    } finally {
        lock.unlock(); // Release the lock
    }
  }
}
```

Cooperation Among Threads

The conditions can be used to facilitate communications among threads. A thread can specify what to do under a certain condition. Conditions are objects created by invoking the newCondition() method on a Lock object. Once a condition is created, you can use its await(), signal(), and signalAll() methods for thread communications. The await() method causes the current thread to wait until the condition is signaled. The signal() method wakes up one waiting thread, and the signalAll() method wakes all waiting threads.



Causes the current thread to wait until the condition is signaled.
Wakes up one waiting thread.
Wakes up all waiting threads.

The lock configuration

```
Lock lock = new ReentrantLock();
Condition notFull = lock.newCondition();
Condition notEmpty = lock.newCondition();
```

The shared data

```
Object[] buffer = new Object[100];
int count = 0, putPtr = 0, takePtr = 0;
```

The write method can only write if buffer not full

```
public void put(Object item) throws InterruptedException
{
 lock.lock();
 try {
  while (count == buffer.length) {
  notFull.await(); // wait until there's space
 }
 buffer[putPtr] = item;
 putPtr = (putPtr + 1) % buffer.length;
 count++;
 notEmpty.signal(); // signal that buffer is not empty
 } finally {
  lock.unlock();
 }
}
```

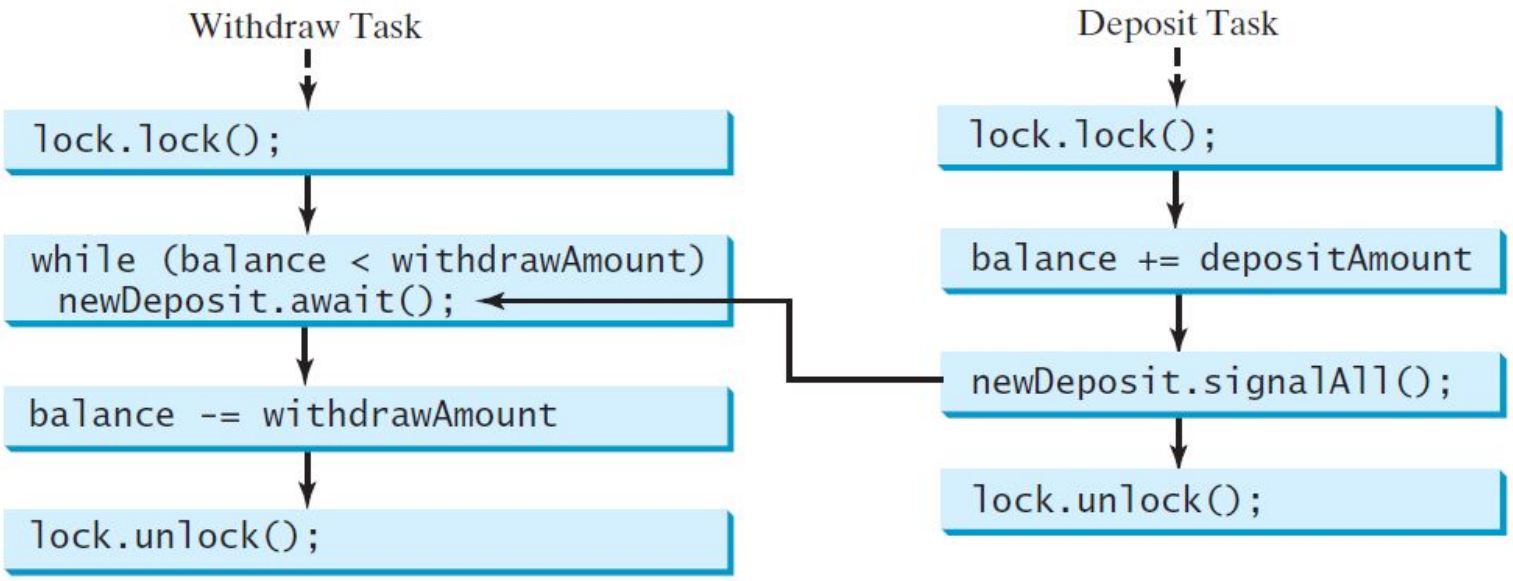
The read method can only read if buffer not empty

```
public Object take() throws InterruptedException {
 lock.lock();
 try {
 while (count == 0) {
  notEmpty.await(); // wait until there's an item
 }
 Object item = buffer[takePtr];
 takePtr = (takePtr + 1) % buffer.length;
 count--;
 notFull.signal(); // signal that buffer is not
full
 return item;
 } finally {
  lock.unlock();
 }
}
```

Cooperation Among Threads

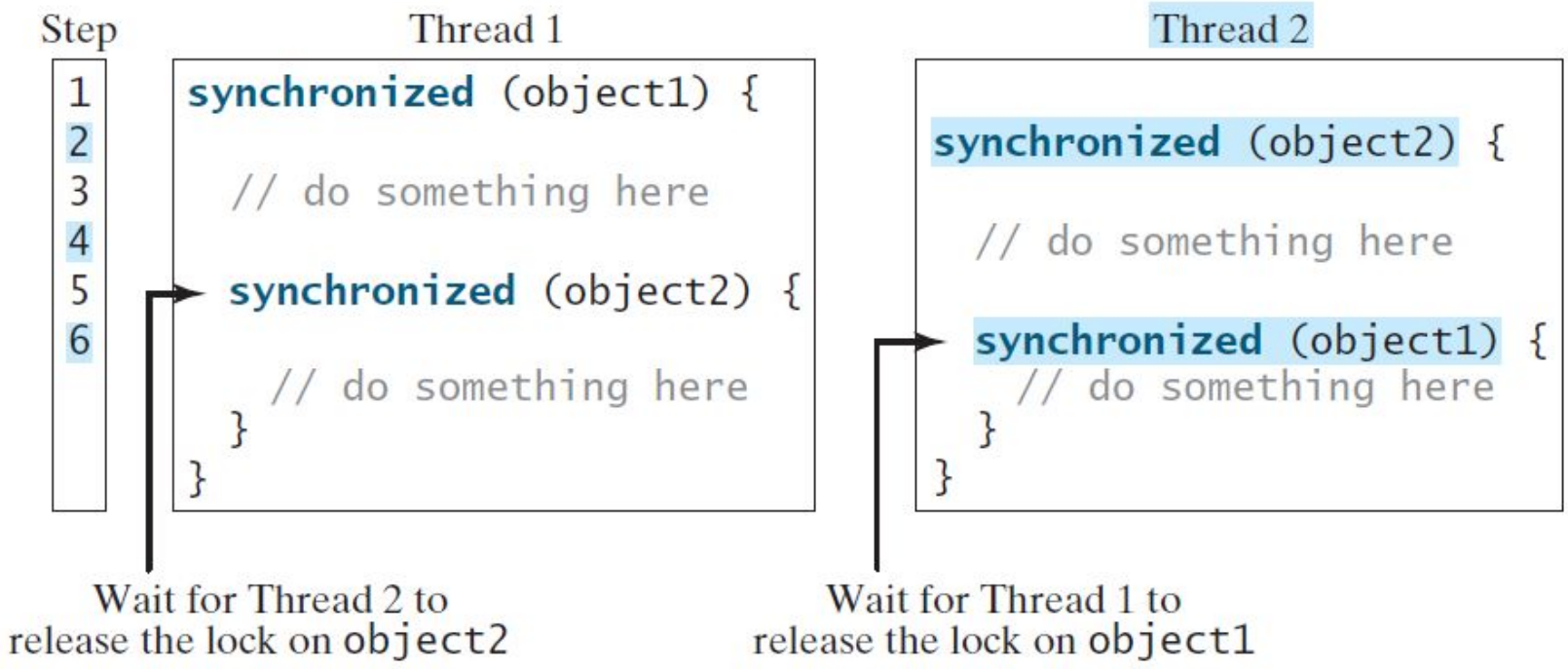
To synchronize the operations, use a lock with a condition: newDeposit (i.e., new deposit added to the account). If the balance is less than the amount to be withdrawn, the withdraw task will wait for the newDeposit condition.

When the deposit task adds money to the account, the task signals the waiting withdraw task to try again.



Deadlock

Sometimes two or more threads need to acquire the locks on several shared objects. This could cause *deadlock*, in which each thread has the lock on one of the objects and is waiting for the lock on the other object. Consider the scenario with two threads and two objects. Thread 1 acquired a lock on object1 and Thread 2 acquired a lock on object2. Now Thread 1 is waiting for the lock on object2 and Thread 2 for the lock on object1. The two threads wait for each other to release the in order to get the lock, and neither can continue to run.



Preventing Deadlock

Deadlock can be easily avoided by using a simple technique known as resource ordering. With this technique, you assign an order on all the objects whose locks must be acquired and ensure that each thread acquires the locks in that order.

For example, suppose the objects are ordered as object1 and object2. Using the resource ordering technique, Thread 2 must acquire a lock on object1 first, then on object2.

Once Thread 1 acquired a lock on object1, Thread 2 has to wait for a lock on object1. So Thread 1 will be able to acquire a lock on object2 and no deadlock would occur.

Preventing Deadlock

When you impose a **global, consistent ordering** on lock acquisition, you eliminate the possibility of circular wait, one of the four conditions required for deadlock.

Suppose you assign every lock an order:

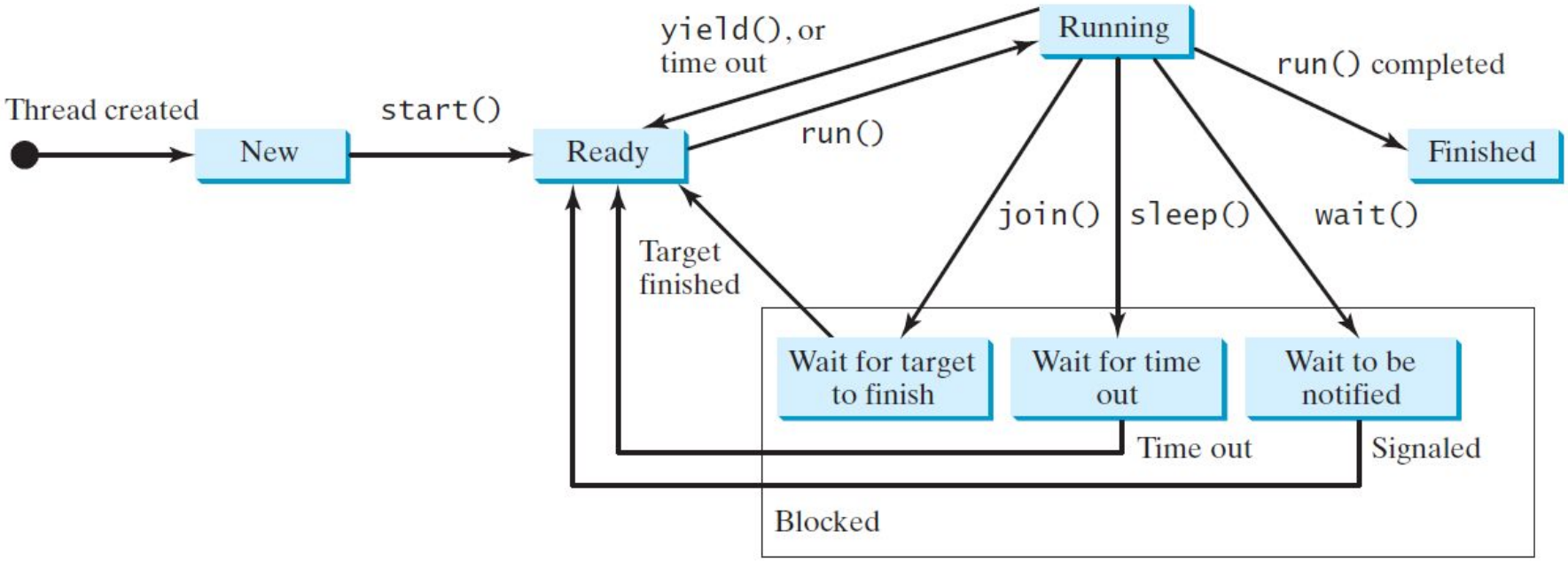
Lock A < Lock B < Lock C

Every thread, no matter what it's doing, must acquire locks in that order.

Even if you have 1000 threads, none of them can create a cycle.

Thread States

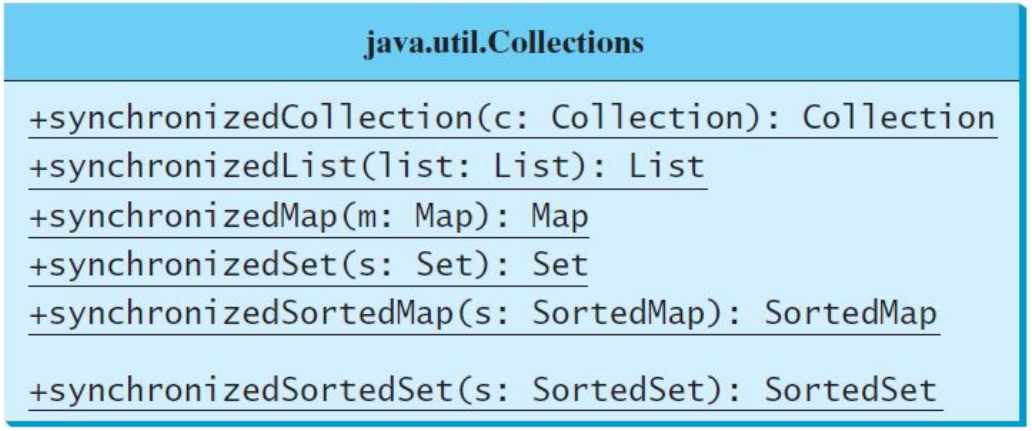
A thread can be in one of five states: New, Ready, Running, Blocked, or Finished.



Synchronized Collections

The classes in the Java Collections Framework are not thread-safe, i.e., the contents may be corrupted if they are accessed and updated concurrently by multiple threads. You can protect the data in a collection by locking the collection or using synchronized collections.

The Collections class provides six static methods for wrapping a collection into a synchronized version. The collections created using these methods are called *synchronization wrappers*.



Returns a synchronized collection.	
Returns a synchronized list from the specified list.	
Returns a synchronized map from the specified map.	
Returns a synchronized set from the specified set.	
Returns a synchronized sorted map from the specified sorted map.	
Returns a synchronized sorted set.	

Vector, Stack, and Hashtable

Invoking synchronizedCollection(Collection c) returns a new Collection object, in which all the methods that access and update the original collection c are synchronized. These methods are implemented using the synchronized keyword. For example, the add method is implemented like this:

```
public boolean add(E o) {
  synchronized (this) { return c.add(o); }
}
```

The synchronized collections can be safely accessed and modified by multiple threads concurrently.

The methods in java.util.Vector, java.util.Stack, and Hashtable are already synchronized. These are old classes introduced in JDK 1.0.

In JDK 1.5, you should use java.util.ArrayList to replace Vector, java.util.LinkedList to replace Stack, and java.util.Map to replace Hashtable. If synchronization is needed, use a synchronization wrapper.

Fail-Fast

The synchronization wrapper classes are thread-safe, but the iterator is *fail-fast*. This means that if you are using an iterator to traverse a collection while the underlying collection is being modified by another thread, then the iterator will immediately fail by throwing java.util.ConcurrentModificationException, which is a subclass of RuntimeException. To avoid this error, you need to create a synchronized collection object and acquire a lock on the object when traversing it. For example, suppose you want to traverse a set, you have to write the code like this:

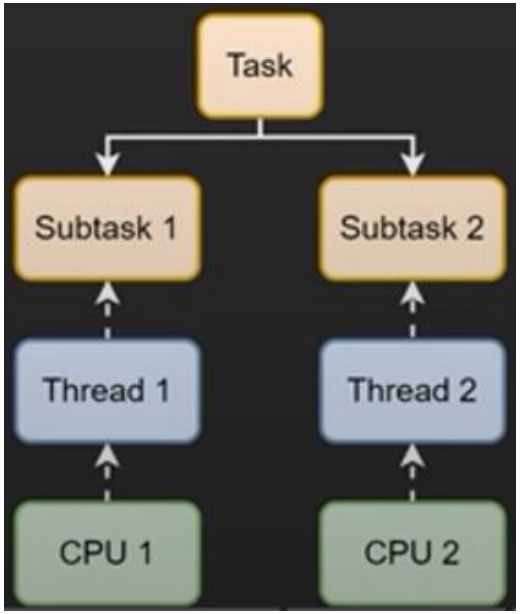
```
Set hashSet = Collections.synchronizedSet(new HashSet());
synchronized (hashSet) { // Must synchronize it
  Iterator iterator = hashSet.iterator();
  while (iterator.hasNext()) {
    System.out.println(iterator.next());
  }
}
```

Failure to do so may result in nondeterministic behavior, such as ConcurrentModificationException.

The Fork/Join multi-threading Framework

The widespread use of multicore systems has created a revolution in software. In order to benefit from multiple processors, software needs to run in parallel.

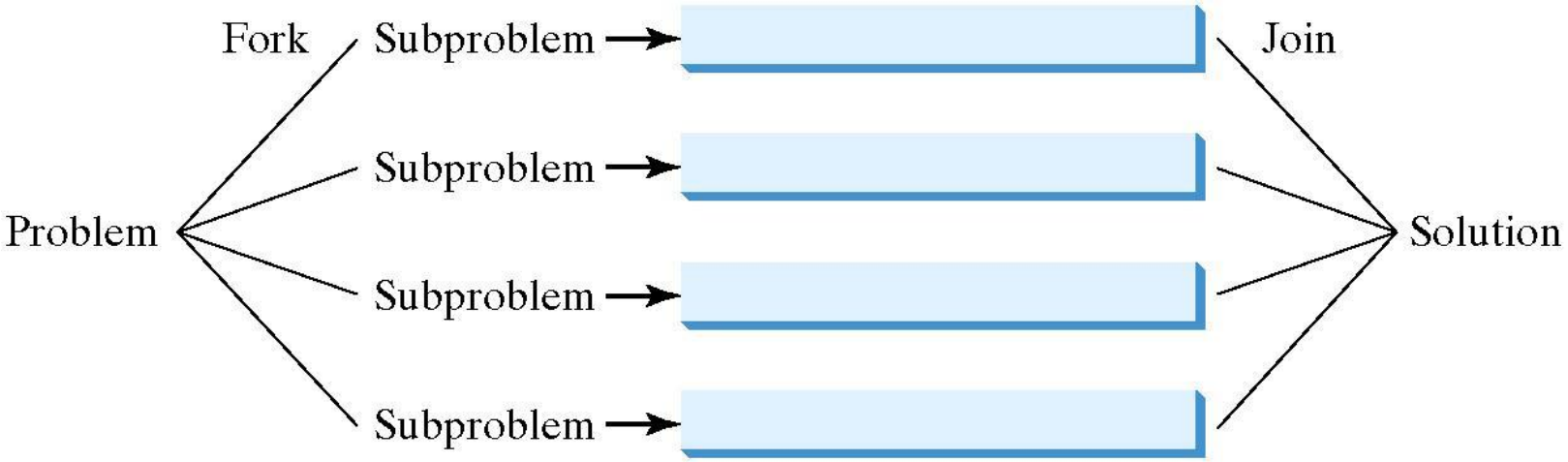
JDK 7 (2011) introduced the new Fork/Join Framework for parallel programming, which utilizes the multicore processors.



The Fork/Join Framework

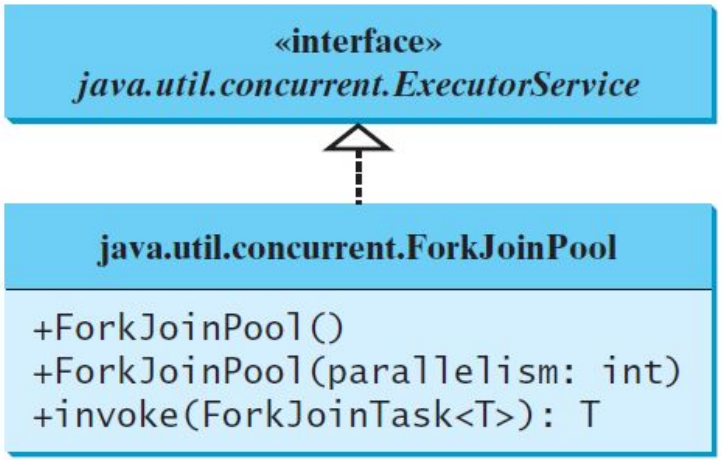
The Fork/Join Framework is used for parallel programming in Java. Every processor has a separate thread queue.

In JDK 7's Fork/Join Framework, a *fork* can be viewed as an independent task that runs on a thread.



ForkJoinTask and **ForkJoinPool**

The framework defines a task using the **ForkJoinTask** class, and executes a task in an instance of **ForkJoinPool**.



See Figure 30.7

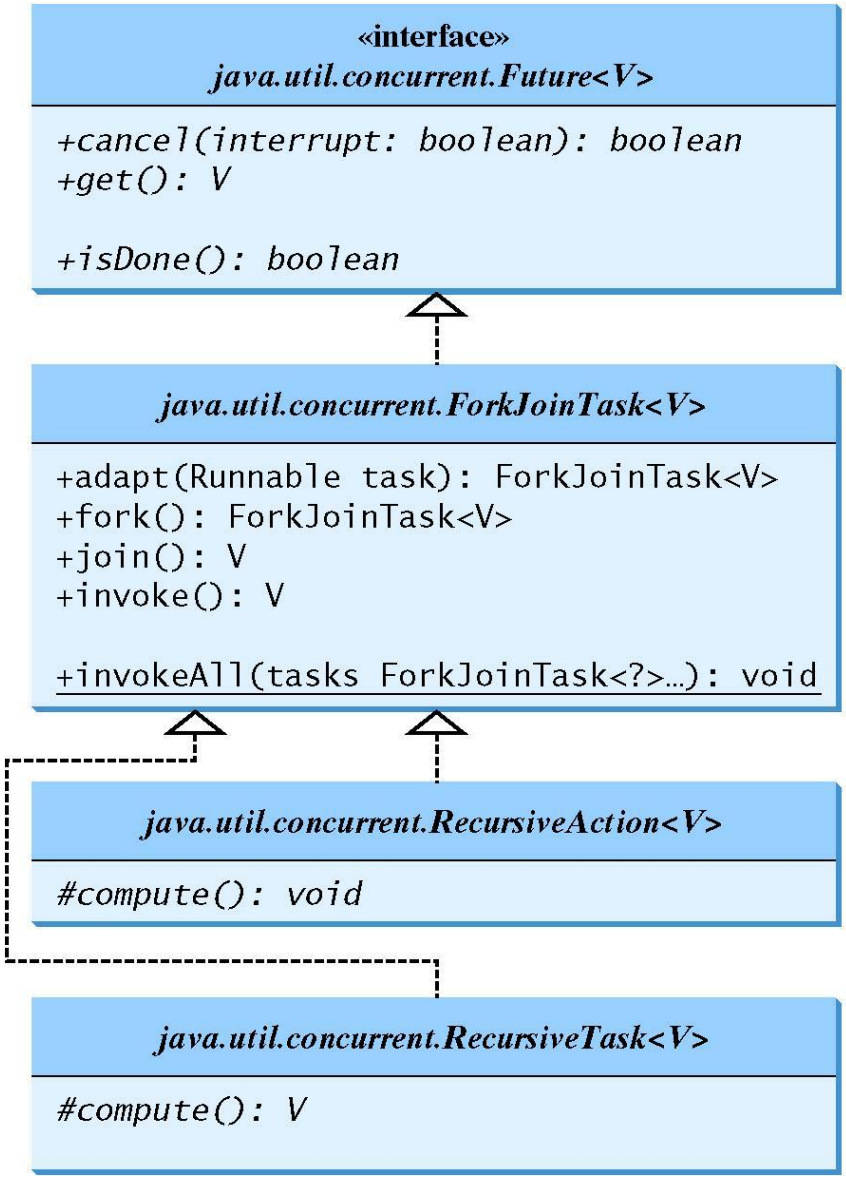
Creates a `ForkJoinPool` with all available processors.
Creates a `ForkJoinPool` with the specified number of processors.
Performs the task and returns its result upon completion.

The *ForkJoinPool* is the heart of the framework. It is an implementation of the *ExecutorService* that manages worker threads and provides us with tools to get information about the thread pool state and performance:

```
// 6 parallel threads max
ForkJoinPool commonPool = new ForkJoinPool(6);
```

ForkJoinTask

ForkJoinTask is the base type for tasks executed inside *ForkJoinPool*.



Attempts to cancel this task.
Waits if needed for the computation to complete and returns the result.
Returns true if this task is completed.

Abstract

Returns a `ForkJoinTask` from a runnable task.
Arranges asynchronous execution of the task.
Returns the result of computations when it is done.
Performs the task and awaits for its completion, and returns its result.
Forks the given tasks and returns when all tasks are completed.

Concrete

Defines how task is performed.

Concrete

Defines how task is performed. Return the value after the task is completed.

Example: Fork

```
Parallel time with 6 processors is 525 milliseconds

Sequential time is 1180 milliseconds
```

(6 cores PC)

```
Parallel time with 20 processors is 321 milliseconds

Sequential time is 767 milliseconds
```

(20 cores PC)

```
public static void main(String[] args) {
  final int SIZE = 7000000;
  int[] list1 = new int[SIZE];
  int[] list2 = new int[SIZE];

  for (int i = 0; i < list1.length; i++)
    list1[i] = list2[i] = (int)(Math.random() * 10000000);

  long startTime = System.currentTimeMillis();
  parallelMergeSort(list1); // Invoke parallel merge sort
  long endTime = System.currentTimeMillis();
  System.out.println("\nParallel time with " + Runtime.getRuntime().availableProcessors() +
    " processors is " + (endTime - startTime) + " milliseconds");

  startTime = System.currentTimeMillis();
  MergeSort.mergeSort(list2); // MergeSort is in Listing 24.5
  endTime = System.currentTimeMillis();
  System.out.println("\nSequential time is " + (endTime - startTime) + " milliseconds");
}

public static void parallelMergeSort(int[] list) {
  RecursiveAction mainTask = new SortTask(list);
```

Example Fork (2)

```
private static class SortTask extends RecursiveAction {
  private final int THRESHOLD = 500;
  private int[] list;

  SortTask(int[] list) { this.list = list;}

  @Override
  protected void compute() {
    if (list.length < THRESHOLD) java.util.Arrays.sort(list);
    else {
      // Obtain the first half
      int[] firstHalf = new int[list.length / 2];
      System.arraycopy(list, 0, firstHalf, 0, list.length / 2);

      // Obtain the second half
      int secondHalfLength = list.length - list.length / 2;
      int[] secondHalf = new int[secondHalfLength];
      System.arraycopy(list, list.length / 2,
        secondHalf, 0, secondHalfLength);

      // Merge firstHalf with secondHalf into list
      // Recursively sort the two halves, does the fork and wait (join)
      MergeSort.merge(firstHalf, secondHalf, list);
      invokeAll(new SortTask(firstHalf),
    }    new SortTask(secondHalf));
  }
}
```